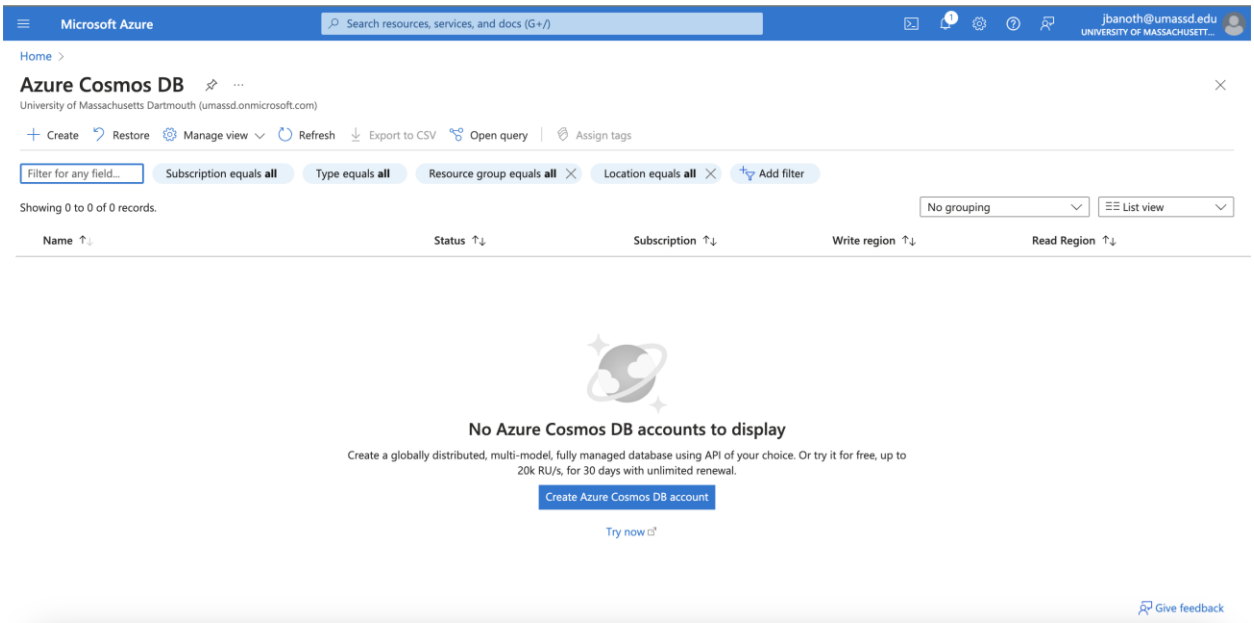
Jeevan Kumar Banoth – 02105145

CIS 552: Database Design

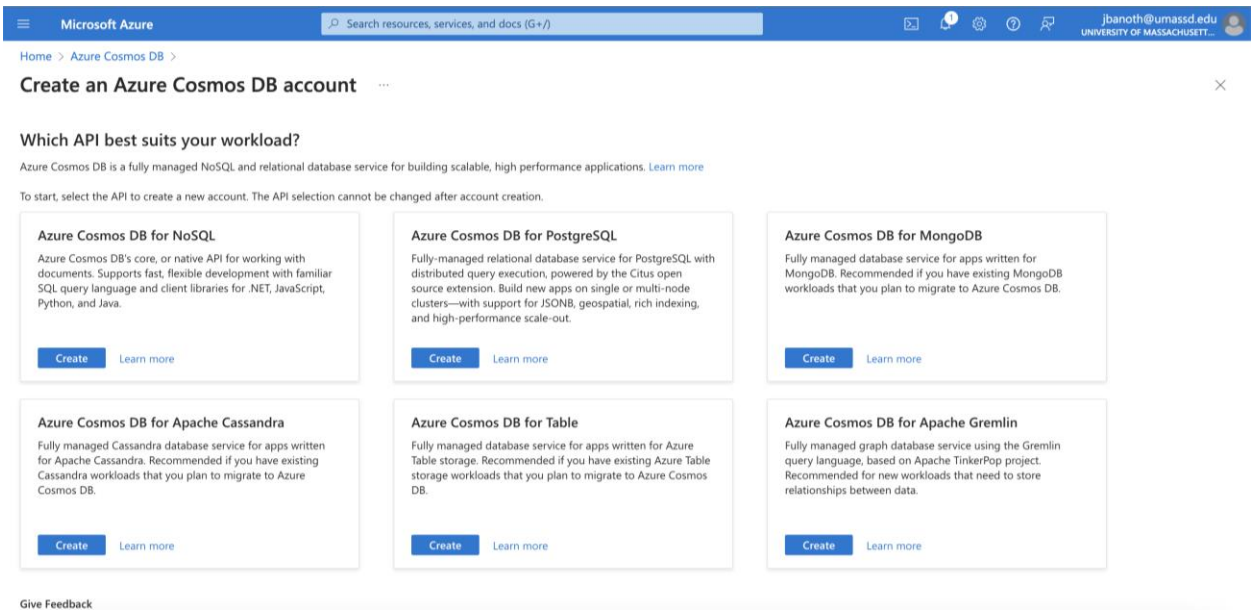
Homework – 5:

Step 1: Amongst other things, we need to create an account for Microsoft Azure Cosmos DB

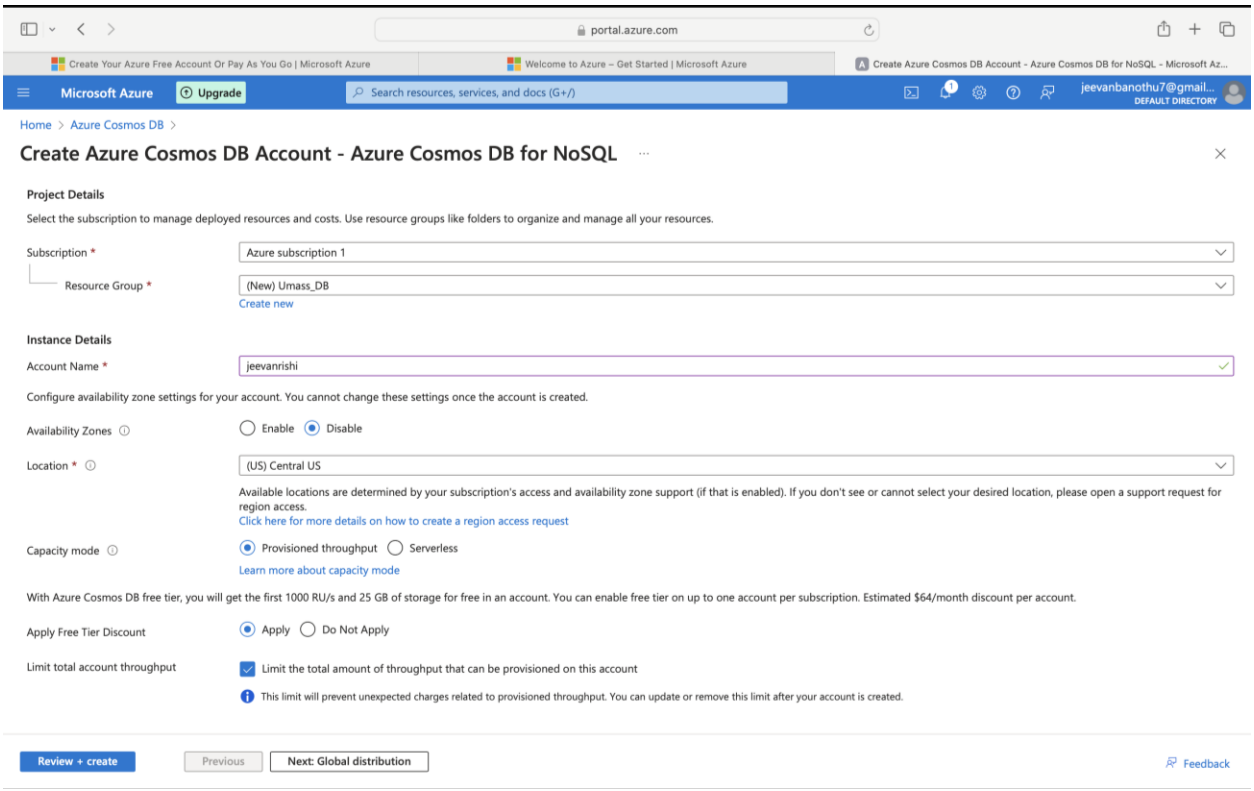
From the below image which shows that Azure Cosmos DB has been created, there is no DB hence the need for a new one



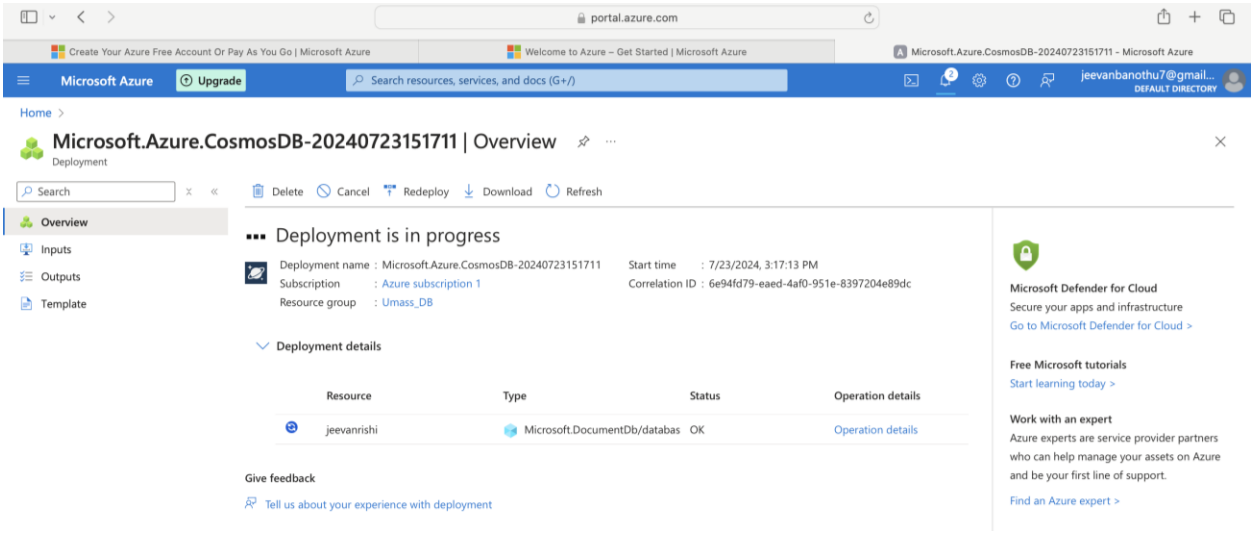
This stage requires us to choose Azure Cosmos DB for NoSQL and to continue with additional information.



Now, we will be creating the Cosmos DB for NoSQL as below and next, review and creat then we will have the Cosmos Database ready in some time.

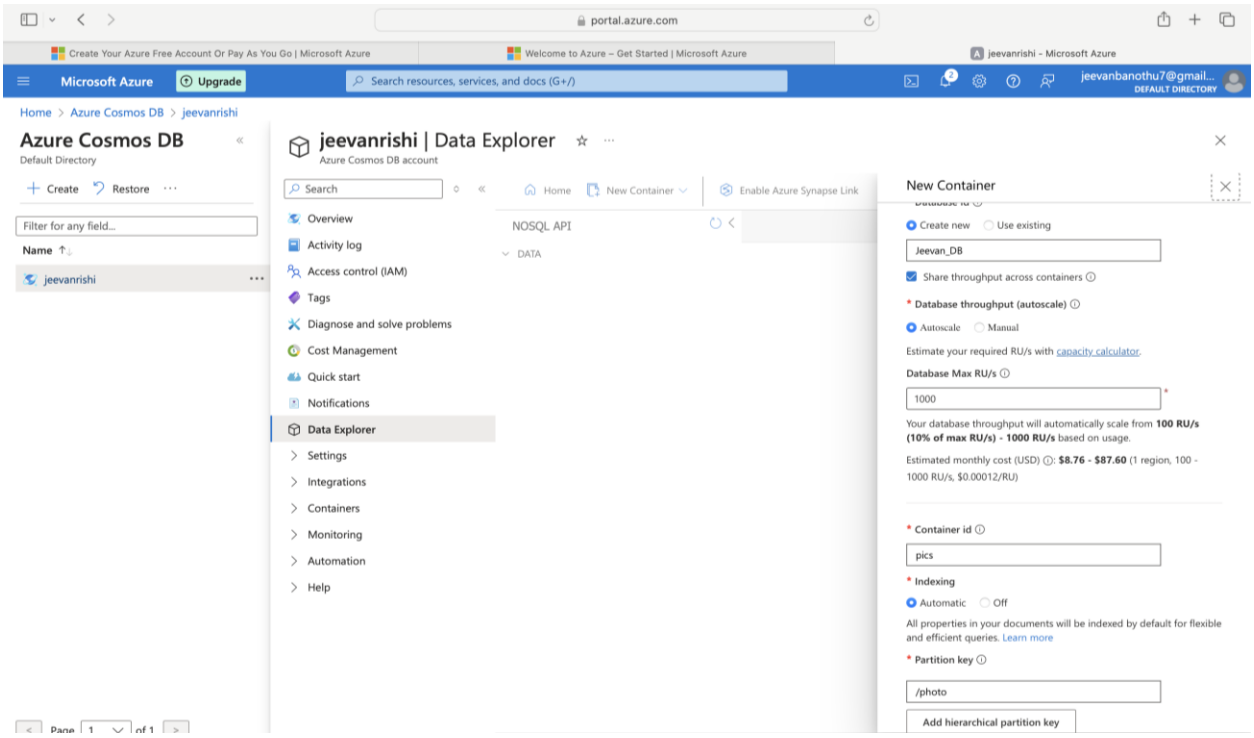


This below Image is the deployment of one, it will take some time and then we are ready to go.



Step 2: Creating a Sample Container:

The container is somewhat like a SQL database table and saves JSON documents. We have set the container identifier “music,” which can symbolize a group of music-related files, and a partition key “/artist,” which divides the data into various parts for scalability.



When you are selecting a partition key, it is essential to select an attribute which has a large variety of values and frequently acts as a filter in the queries.

This ensures that information is evenly spread among departments and makes queries as well as transactions operate more efficiently. If your queries usually use ‘player’ as one of their filters while your data set contains many different players, then this could be good candidate for disk key.

Step 3: Establishing Container Entities:

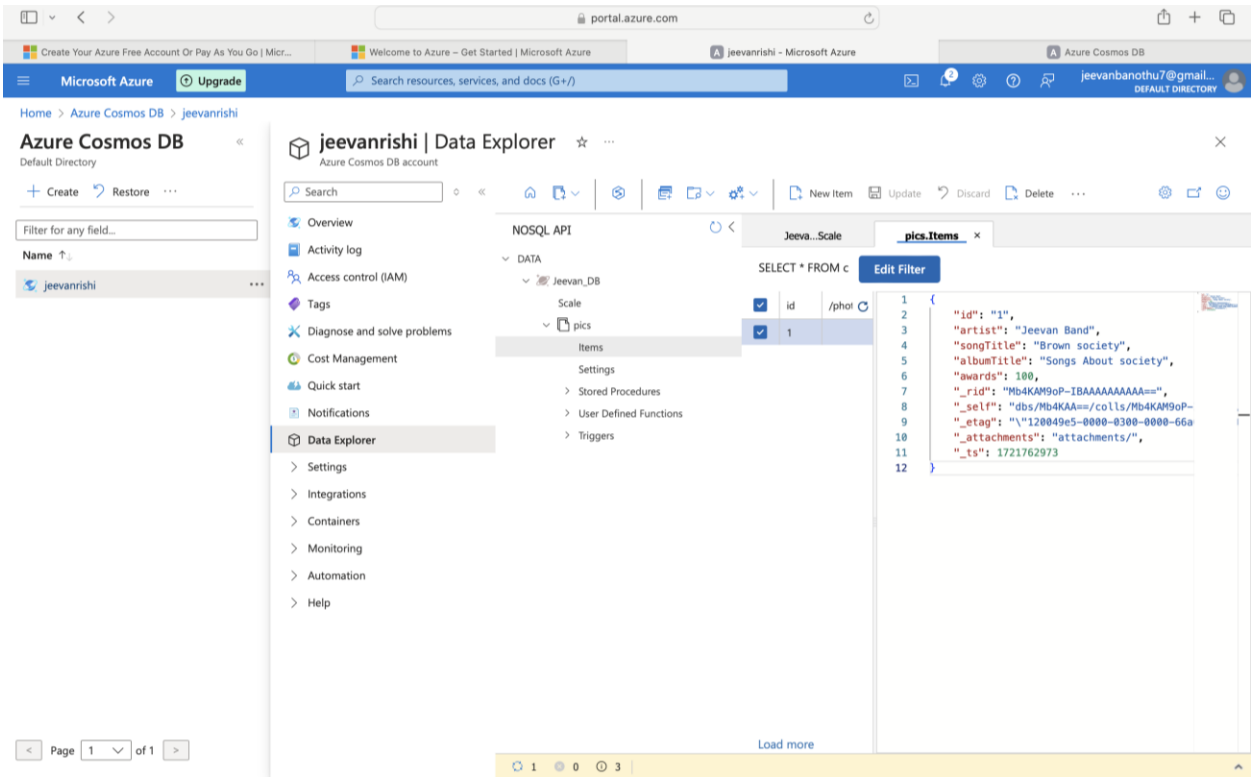
Currently, we are using the insert tab to add additional elements to the music.

id: A distinct identification.

artist: "Jeevan Band" is the name of the artist.

song title: "Brown Society" is the song's title.

"Songs About Society" is the title of the album.



honors: The song has received ten accolades overall.

id: The container's unique identifier, which is set to "1".

Artist: The name of the artist (in this example, "Arijit Singh") connected to the query.

SongTitle: The song's title, which is "Chaiyya" in this case.

Fifteen awards have been given to this song or performer.

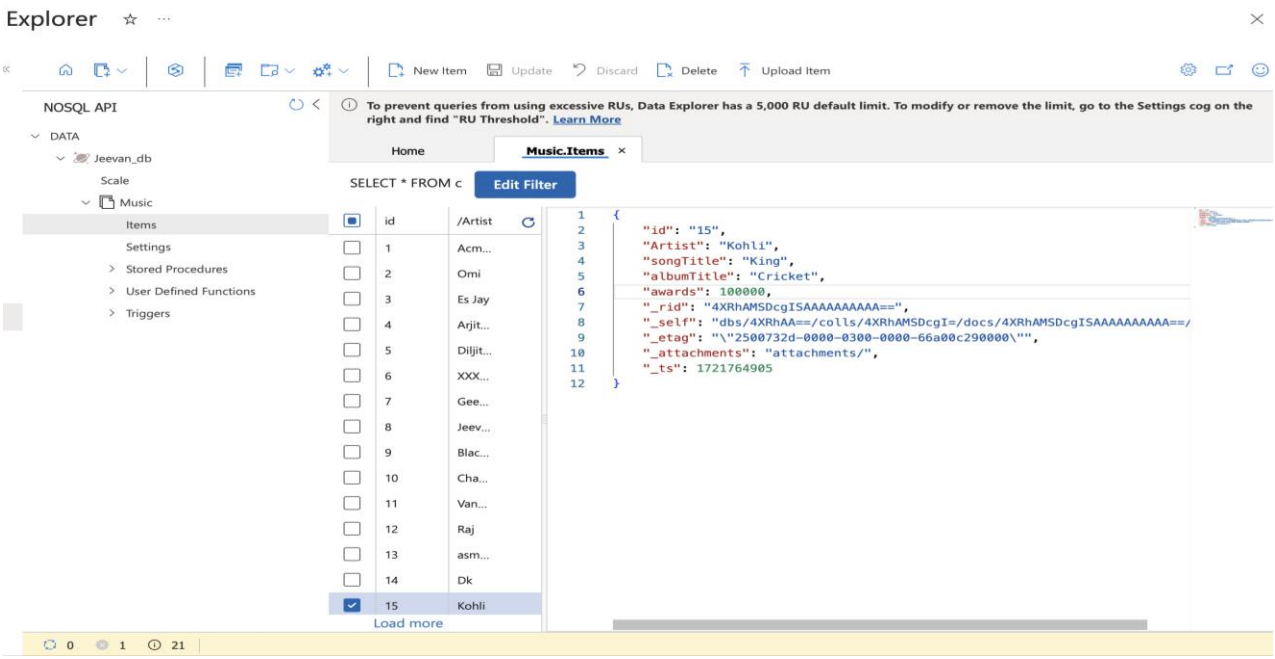
removed Resource ID: a unique internal number assigned to each resource in Azure Cosmos DB

self: The Azure Cosmos DB resource's Uniform Resource Locator (URI), which is used to obtain the resource via the REST API.

An eTag is an entity tag used for optimistic concurrency control. Users are less likely to inadvertently overwrite changes made by other users.

_attachments: This is only a placeholder for where any attachments related to the query may be found.

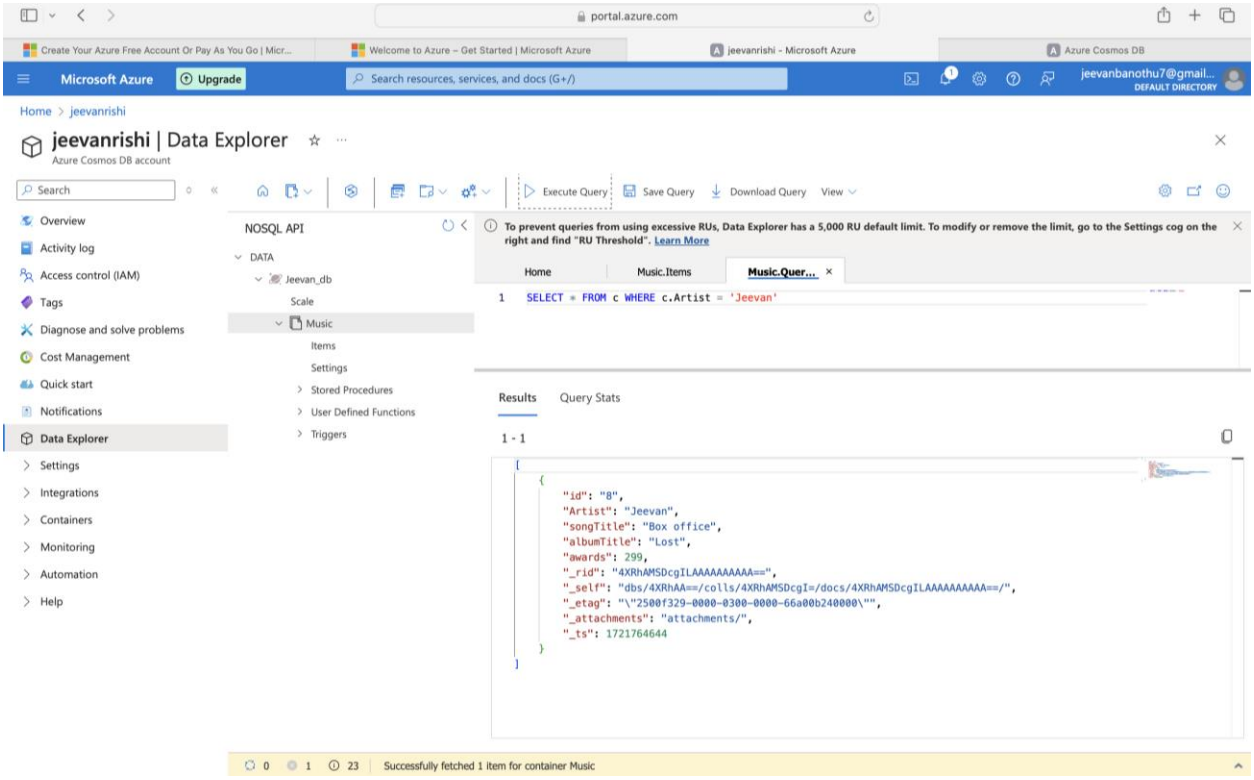
_ts: The most recent update is shown by this timestamp. The number 1711499141 denotes the number of seconds that have elapsed since the Unix epoch, which occurred on January 1, 1970.



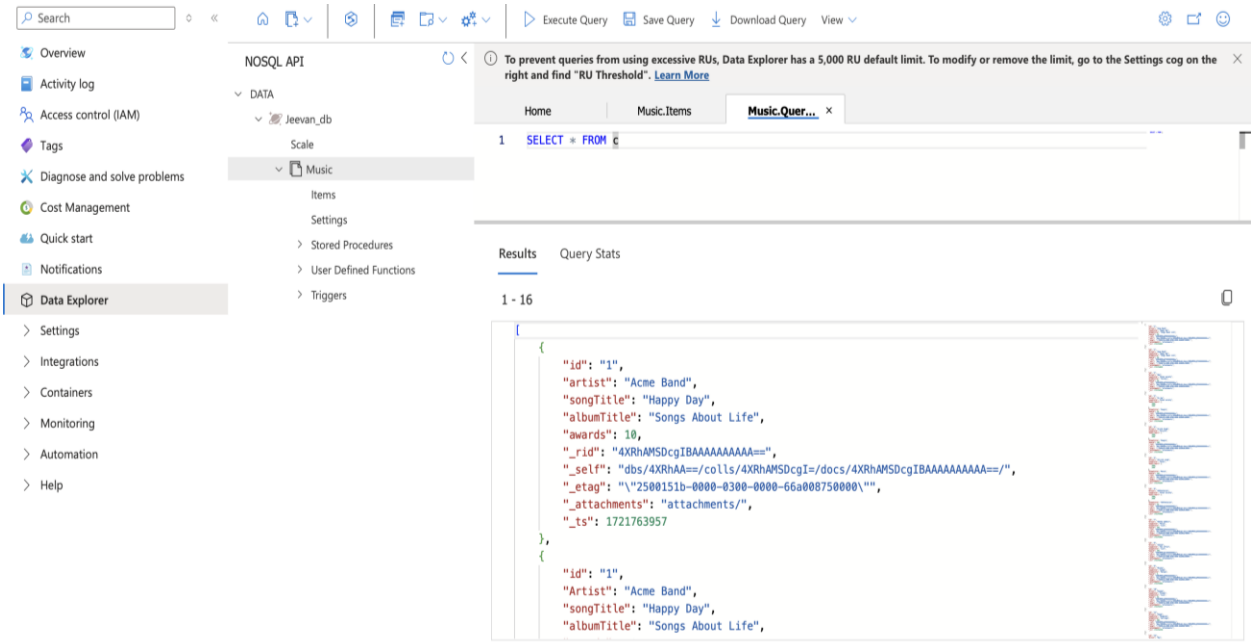
Lines, they are as diverse as love letters, representing documents within the container. Accompanied by a unique ID as well as sender names, each line corresponds with a specific document in the collection.

The artist connected to a text with document ID of 3 is “Arijit Singh”. This view is sorted by identifiers of individual documents which appear like in the table below either being automatically or manually allocated such sequential numbers for distinguishing them from one another.

Step:4: Querying:



A QL-like query is invoked to extract every document from 'the music bin' (or 'c'), in which the artist field has 'Jeevan' as its value. id: Unique identifier for this document belonging to that container; value "8". Artist: An artist's name is given as "Jeevan". Song Title: Title of the song, i.e., "Box office". The number of awards listed is 299.

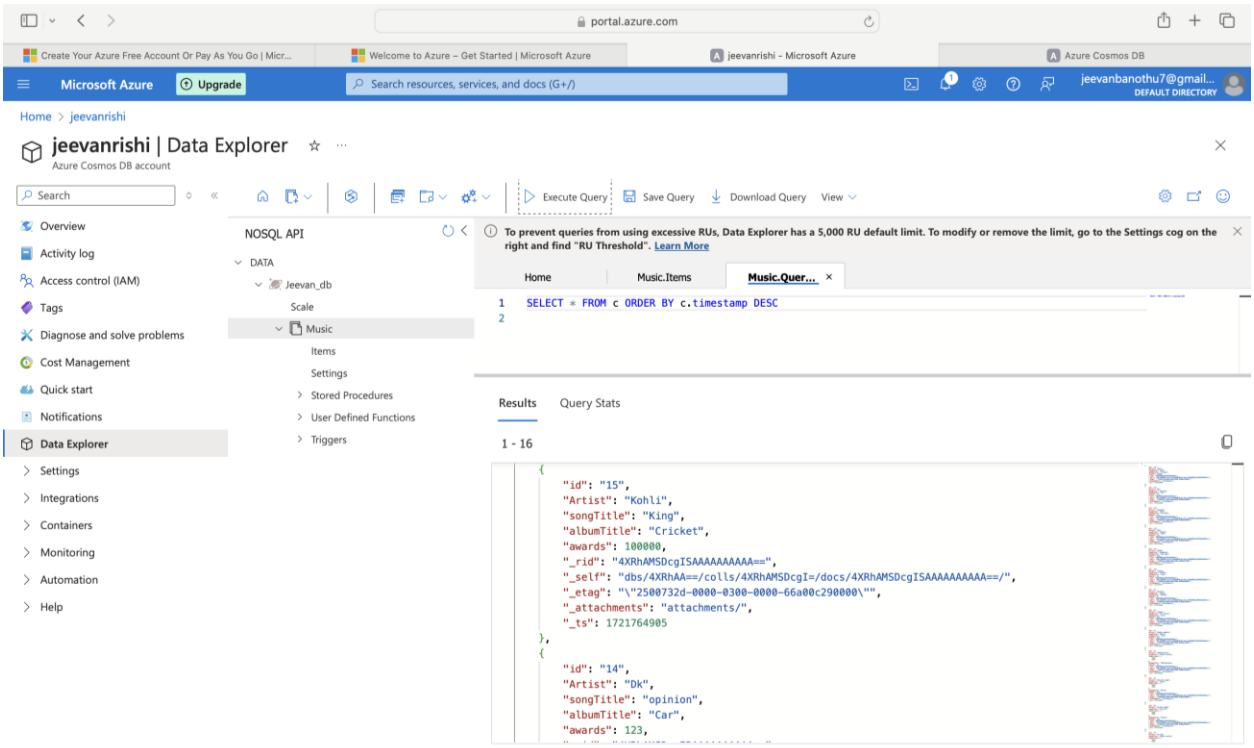


Every field in the documents must be returned, according to a SQL-like query that gets every document from the sample database music box.

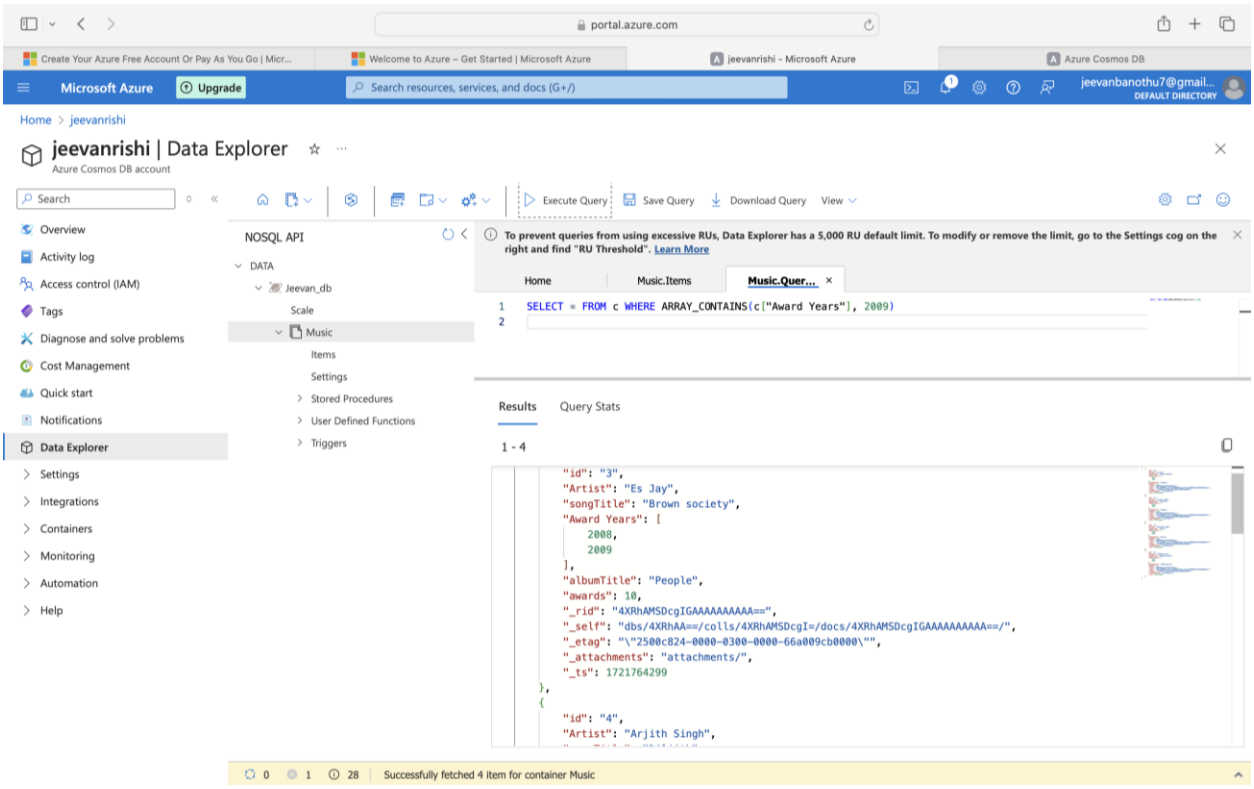
ID: "1"

Artist: "Artist Name" Song title: "Song Title" Album title: "Album Title" Issued in 2020

Category: "Genre"

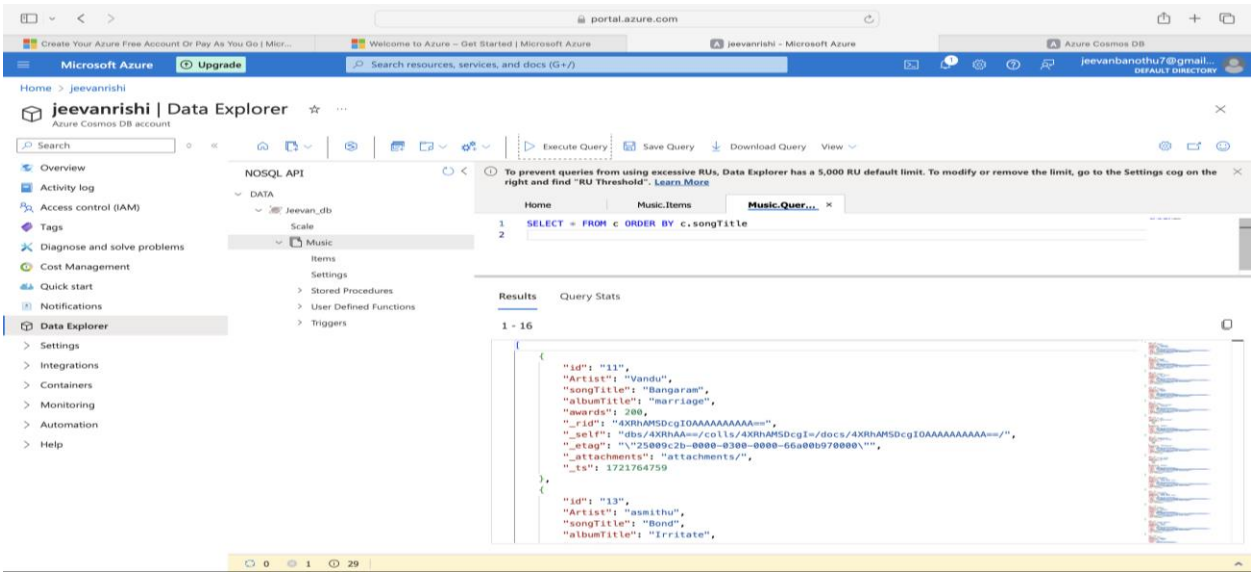


You can order documents based on a given field using the Cosmos DB Core (SQL) API, which often has query syntax like SQL. Here, c stands for the container's alias, and timestamp is a required field in all documents that contains the creation or last update time.



Checks whether the array c ['Award Years'] has the value 2009 using the ARRAY_CONTAINS function.

The artist "Taylor Swift" and the track "Love Story" are identified by the viewable document id, which is "1". This song won awards in both years, included in the award year table, 2009 and 2010.



For papers with IDs less than 4, the complete paragraph name field is displayed; for documents with IDs greater than 4, only the ID field is displayed. Take "Love Story" as an example, denoted by id 4, and "Bad Romance" by id 2.

Conclusion:

In this lab, we deftly maneuvered around setting up and using an entirely managed NoSQL database service known as Microsoft Azure Cosmos DB. The lab took us through how to use Cosmos DB beginning with the creation of a Cosmos DB account.

Upon initialization, we spotted a section asking for a new database instance to be created. Thereafter, we moved ahead to create a music container within our sample database with "music" as its container ID and "/artist" as its section.

The key This is meant to ensure that both queries and transactions perform better since they will not always be directed towards the same set of data; it also signifies the heightening dimensions of scalability within Cosmos DB.

When consuming data from the music repository, we leveraged on the flexible model of Cosmos DB. adapt to different musical entities as an artist. name, canton name, album name, awards, and year of release.

These characteristics show off the schema-free nature of Cosmos DB which allows us to manage diverse data architectures in one container.

By executing a sequence of SQL like commands on basic datasets we illustrated how powerful examples for querying functions are using some queries in CosmosDB which are mainly limited by category-based filters or even personalized timestamps available through sorting results alphabetically.

Overall, this interactive lab gave participants a thorough understanding of Azure Cosmos DB and demonstrated how well it manages NoSQL data in addition to having a strong query engine that enables scalable, dependable support for intricate tasks.